



Training Python-Django - 0

OOB

Summary: Today, you're going to conquer Silicon Valley with your newly acquired skills in POO with Python!

Version: 1.2

Contents

I	Preamble	2
II	General rules	3
III	Today's specific rules	4
IV	Exercise 00	5
V	Exercise 01	7
VI	Exercise 02	8
VII	Exercise 03	10
VIII	Exercise 04	12
IX	Exercise 05	14
X	Exercise 06	16
XI	Submission and peer-evaluation	18

Chapter I

Preamble

Here are the lyrics of the "Free Software Song":

Join us now and share the software;
You'll be free, hackers, you'll be free.
Join us now and share the software;
You'll be free, hackers, you'll be free.

Hoarders can get piles of money,
That is true, hackers, that is true.
But they cannot help their neighbors;
That's not good, hackers, that's not good.

When we have enough free software
At our call, hackers, at our call,
We'll kick out those dirty licenses
Ever more, hackers, ever more.

Join us now and share the software;
You'll be free, hackers, you'll be free.
Join us now and share the software;
You'll be free, hackers, you'll be free.

Chapter II

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a 0 during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter III

Today's specific rules

- No code in the global scope. We want functions!
- Unless otherwise specified, every file written in Python will have to end with a block

```
if __name__ == '__main__':  
    # Your tests and your error handling
```

- Any uncaught exception will negate the work, even in case of an error you were required to test.
- Any import is prohibited, except for the ones specified in the 'Authorized functions' section of each exercise's header.
- You'll have to use the `python3` interpreter.

Chapter IV

Exercise 00

	Exercise 00
Exercise 00 : Conquering Silicon Valley!	
Turn-in directory : <i>ex00/</i>	
Files to turn in : <code>render.py</code> , <code>myCV.template</code> , <code>settings.py</code>	
Allowed functions : <code>import sys, os, re</code>	

You've just completed your developer training course, and new perspectives are about to change your life forever. You arrive in Silicon Valley with one goal in mind: develop your revolutionary resume generator using a groundbreaking technology and become a major figure in the job-seeking industry!

Now all that's left to do is... craft your technology.

Create a `render.py` program that will take a file with a `.template` extension as a parameter. This program must read the file's content, replace some patterns with values defined in a `settings.py` file (an `if __name__ == '__main__'` block will not be necessary for this file), and write the result into a file with a `.html` extension.

You *will have* to be able to replicate the following example with your program:

```
$> cat settings.py
name = "duoquadragintian"
$> cat file.template
<p>"Who are you?
-A {name}!"</p>
$> python3 render.py file.template
$> cat file.html
<p>"Who are you?
-A duoquadragintian!"</p>
```

Errors, including wrong file extension, non-existing file, or wrong number of arguments, must be handled.

You will have to turn in a `myCV.template` file which, once converted to HTML, must include at least the full structure of a page (`doctype`, `head`, and `body`), the page title, the name and surname of the resume owner, their age, and profession. Of course, this information must not directly appear in the `.template` file.



`help(globals)`, keyword expansion...

Chapter V

Exercise 01

	Exercise 01
Exercise 01: Innovating start-up looking for intern. 10 years experience required.	
Turn-in directory : <i>ex01/</i>	
Files to turn in : intern.py	
Allowed functions : None	

You cannot start this journey alone. You decide to recruit someone to ~~make coffee~~, assist you; an intern would be better (they're cheaper).

Create the **Intern** class containing the following functionalities:

A **constructor** taking a string as a parameter and assigning its value to a **Name** attribute. The default value must be "My name? I'm nobody, an intern, I have no name.".

A **method `__str__()`** that returns the **Name** attribute of the instance.

A **Coffee class** with a simple **`__str__()`** method that returns the string: "This is the worst coffee you ever tasted.".

A **`work()` method** that raises a single exception (using the base **Exception** type) with the message: "I'm just an intern, I can't do that...".

A **`make_coffee()` method** that returns an instance of the **Coffee** class defined inside the **Intern** class.

In your tests, you must instantiate the **Intern** class twice: once without a name, and a second time with the name "Mark".

Display the name of each instance. Ask Mark to make you a coffee and display the result. Ask the other intern to work. You **must** handle the exception in your test.

Chapter VI

Exercise 02

	Exercise 02
Exercise 02: 5 classes	
Turn-in directory : <i>ex02/</i>	
Files to turn in : beverages.py	
Allowed functions : None	

Coffee is good. Choosing your drink is better! Create a **HotBeverage** class with the following functionalities:

A **price attribute** with a value of 0.30.

A **name attribute** with the value "hot beverage".

A **description() method** returning an instance description. The description value must be "Just some hot water in a cup."

A **__str__() method** returning an instance description in the following form:

```
name : <name attribute>
price : <price attribute limited to two decimal points>
description : <instance's description>
```

For instance, a **HotBeverage** instance would display:

```
name : hot beverage
price : 0.30
description : Just some hot water in a cup.
```

Then create the following classes derived from **HotBeverage**:

Coffee:

```
name: "coffee"
price: 0.40
description: "A coffee, to stay awake."
```

Tea:

```
name: "tea"  
price: 0.30  
description: "Just some hot water in a cup."
```

Chocolate:

```
name: "chocolate"  
price: 0.50  
description: "Chocolate, sweet chocolate..."
```

Cappuccino:

```
name: "cappuccino"  
price: 0.45  
description: "Un po' di Italia nella sua tazza!"
```



You must only redefine what's necessary. Avoid duplication and follow the DRY principle (cf. [DRY](#)).

In your tests, instantiate each class among `HotBeverage`, `Coffee`, `Tea`, `Chocolate`, and `Cappuccino`, and display them.

Chapter VII

Exercise 03

	Exercise 03
Exercise 03: Glorious coffee machine!	
Turn-in directory : <code>ex03/</code>	
Files to turn in : <code>machine.py</code> , <code>beverages.py</code>	
Allowed functions : <code>import random</code>	

There you are! Your company is up and running! Your first fundraising round has provided you with premises. You have an intern for the coffee and a level 10 green plant at the building entrance to keep everything in order.

Yet everything is not perfect: your intern makes awful coffee, and half a minimum wage is expensive for this mud. The time has come to invest in new equipment that will lead to your personal success!

Create the `CoffeeMachine` class containing:

- A constructor.
- An `EmptyCup` class inheriting from `HotBeverage`, with the name "empty cup", a price of 0.90, and the description "An empty cup?! Gimme my money back!".
Copy the `beverages.py` file from the previous exercise into this exercise folder to use the classes it contains.
- A `BrokenMachineException` class inheriting from `Exception`, with the message "This coffee machine has to be repaired.". This message must be defined in the exception's constructor.
- A `repair()` method that fixes the machine so it can serve hot drinks again.
- A `serve()` method with the following specifications:

Parameters: A single parameter (other than `self`) that will be a class derived from `HotBeverage`.

Return: Randomly, the method returns either an instance of the class provided as parameter or an `EmptyCup` instance.

Obsolescence: The machine is cheap and breaks down after serving 10 drinks.

When out of order: Calling the `serve()` method must raise a `CoffeeMachine.BrokenMachine` until the `repair()` method is called.

Fixing: After calling `repair()`, the `serve()` method works again without raising an exception, until it breaks down again after serving 10 drinks.

In your tests, instantiate the `CoffeeMachine` class. Request various drinks from `beverages.py` and display what you are served until the machine breaks down (you must handle the raised exception). Repair the machine and start again until it breaks down again (handle the exception again).

Chapter VIII

Exercise 04

	Exercise 04
Exercise 04: A basic class ft. RMS.	
Turn-in directory : <i>ex04/</i>	
Files to turn in : elem.py	
Allowed functions : None	

It's now time to work on your web visibility. You'd like to use your newly acquired Python skills to efficiently design HTML content, but you'd like to get advice from a superior being to learn how to do so. You choose to sacrifice your intern to the Gods of programming.

Now that you have a coffee machine, he has become pretty useless... Just torch him...

Saint IGNUcius appears before you to speak a precious revelation:

"HTML elements share almost the same structure (tag, content, attributes). It would be wise to create a class that can assemble all those shared behaviors and specifications, and to use the legacy features of Python to derive this class without having to rewrite the whole thing."

Only then does St. IGNUcius see the Mac you're working on. Scared as hell, he flees without further ado, leaving behind a test file and an incomplete class. In a rush, you complete the **Elem** class (the missing parts are indicated with *[...]*) with the following specifications:

- A constructor taking as parameters the element's name, its HTML attributes, and its type (simple or double tags).
- A `__str__()` method returning the element's HTML code.
- An `add_content()` method allowing content to be added to the end of the element's content.
- An *Exception* subclass within the class.

If the work is done well, you will be able to represent any HTML element and its content with your `Elem` class. Now for the final stretch:

- The `tests.py` file provided in the tarball in the appendix of the subject **must** work properly (no assertion errors, and the test output must explicitly indicate success). Of course, we are not cruel enough to test functionalities that are not explicitly required in this exercise. Hahaha... we're not... really.
- You must also replicate and display the following structure using your `Elem` class:

```
<html>
  <head>
    <title>
      "Hello ground!"
    </title>
  </head>
  <body>
    <h1>
      "Oh no, not again!"
    </h1>
    
  </body>
</html>
```

Chapter IX

Exercise 05

	Exercise 05
Exercise 05: Create your own elements!	
Turn-in directory : <i>ex05/</i>	
Files to turn in : elem.py , elements.py	
Allowed functions : None	

Congratulations! Now you can generate any HTML element and its content. However, generating each element by specifying each attribute at each instantiation is a bit tedious. This is an opportunity to use inheritance to create small classes that are easier to use. Create the following classes derived from the **Elem** class created in the previous exercise:

- `html`, `head`, `body`
- `title`
- `meta`
- `img`
- `table`, `th`, `tr`, `td`
- `ul`, `ol`, `li`
- `h1`
- `h2`
- `p`
- `div`
- `span`
- `hr`
- `br`

Each class constructor will be able to take the content as the first argument, and:

```
print( Html( [Head(), Body()] ) )
```

will display:

```
<html>
  <head></head>
  <body></body>
</html>
```

Be smart: reuse the functionalities you implemented in the `Elem` class. You **must** use inheritance.

Demonstrate how these classes work with a number of tests of your choice, covering all functionalities. After implementing these classes, you will no longer need to specify the name or type of a tag, which is very convenient. You will never have to instantiate the `Elem` class directly again. In fact, from now on, this is prohibited.

To understand the benefits of the derived `Elem` classes compared to direct use of `Elem`, let us take the HTML document structure from the previous exercise. You must replicate it using your new classes.

```
<html>
  <head>
    <title>
      "Hello ground!"
    </title>
  </head>
  <body>
    <h1>
      "Oh no, not again!"
    </h1>
    
  </body>
</html>
```

Way simpler, isn't it? :)

Chapter X

Exercise 06

	Exercise 06
Exercise 06: Validation.	
Turn-in directory : <i>ex06/</i>	
Files to turn in : <code>Page.py</code> , <code>elem.py</code> , <code>elements.py</code>	
Allowed functions : <code>None</code>	

Though you've made remarkable progress, your work still needs a bit of cleanup and more conformity. You like constraints and challenges, so why not impose a norm on the structure of your HTML documents? Start by copying the classes from the previous exercises into this exercise's folder.

Create a `Page` class whose constructor takes as a parameter an instance of a class inheriting from `Elem`. Your `Page` class must implement an `is_valid()` method that returns `True` if all the following rules are satisfied, and `False` otherwise:

- If, in the tree path, a node is not one of the following types: `html`, `head`, `body`, `title`, `meta`, `img`, `table`, `th`, `tr`, `td`, `ul`, `ol`, `li`, `h1`, `h2`, `p`, `div`, `span`, `hr`, `br`, or `Text`, the tree is invalid.
- `Html` must strictly contain a `Head` followed by a `Body`.
- `Head` must contain exactly one `Title`.
- `Body` and `Div` must only contain the following types of elements: `H1`, `H2`, `Div`, `Table`, `U1`, `Ol`, `Span`, or `Text`.
- `Title`, `H1`, `H2`, `Li`, `Th`, and `Td` must contain exactly one `Text` element.
- `P` must only contain `Text`.
- `Span` must contain only `Text` or `P` elements.
- `U1` and `Ol` must contain at least one `Li` and only `Li` elements.

- **Tr** must contain at least one **Th** or **Td** and only **Th** or **Td** elements. **Th** and **Td** must be mutually exclusive.
- **Table** must contain only **Tr** elements and at least one **Tr**.

Your **Page** class must also be able to:

- Display its **HTML** code when an instance is printed. Beware: the **HTML** code must be preceded by a doctype if and only if the root element is of type **Html**.
- Write its **HTML** code to a file using a **write_to_file** method that takes the file name as a parameter. Beware: the **HTML** code written to the file must be preceded by a doctype if and only if the root element is of type **Html**.

Demonstrate how your **Page** class works with a number of tests of your choice that cover all functionalities.

Chapter XI

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Double-check the names of your folders and files to ensure they are correct.



The evaluation process will take place on the evaluated group's computer.